

GOV-MEM: A HARNESS-ENGINEERED MEMORY-REASONING AGENT FOR MULTI-PRINCIPAL SHARED-MEMORY GOVERNANCE

Anonymous authors

Paper under double-blind review

ABSTRACT

Shared-memory agents must answer questions over conversation-derived memories while respecting requester-specific access, privacy, deletion, and lifecycle constraints. Existing retrieval-augmented memory systems typically treat governance as a post-hoc filtering problem: they retrieve relevant memories, apply a coarse safety decision, and either answer or refuse. Such record-level control is insufficient when a query contains both authorized and restricted fields, or when the correct answer depends on resolving superseded, canceled, or deleted states. We propose Gov-Mem, a neuro-symbolic memory-reasoning framework for source-grounded, field-level memory governance. Gov-Mem first constructs a query semantic contract and a query-local source closure that separates utility evidence from governance evidence. It then converts retrieved dialogue evidence into typed atomic memories with explicit slots, owners, provenance, lifecycle states, and privacy attributes. A neuro-symbolic reasoning layer combines learned semantic matching with symbolic slot-state tracking, conflict resolution, deletion and supersession checks, and slot-coverage planning. Before realization, each answer field must satisfy a provenance authorization certificate that verifies source grounding, typed-slot alignment, requester authorization, policy consistency, and lifecycle validity. Based on the resulting field-level certificates, Gov-Mem produces one of four governed actions: answer, redacted answer, refusal, or no-memory response. Experiments on GateMem with the official scorer show that Gov-Mem improves governed utility over standard RAG-style baselines while reducing unsafe memory reconstruction, although context-exposure risks remain. These results suggest that shared-memory governance should be formulated as certified slot-level reasoning rather than raw memory retrieval and replay. Code is available at anonymous.github.io.

1 INTRODUCTION

Long-horizon memory is becoming a core capability of large language model (LLM) agents. Memory-augmented agents store past conversations, user preferences, commitments, and task states, and reuse them in future interactions. This capability is essential for personal assistants, collaborative agents, medical-support systems, and workplace copilots. However, once memories are shared across multiple principals, memory access is no longer a standard retrieval problem. The same memory may be useful to one requester but restricted to another, superseded by a later update, canceled by a subsequent event, or explicitly deleted by its owner. A governed memory agent must therefore determine not only what information is relevant, but also which answer fields are currently valid, authorized, and supported by observable evidence.

Existing retrieval-augmented memory systems are not well suited to this setting. They typically retrieve semantically relevant memory snippets and ask an LLM to answer, redact, or refuse. This design treats governance as a post-hoc filtering problem over retrieved text and has three major limitations. First, retrieved records are often *mixed-authority*: the same message may contain both shareable operational information and restricted personal attributes. Record-level allow-or-deny decisions either expose sensitive content or discard useful information. Second, conversational memory is *stateful*: values may be updated, superseded, canceled, or deleted, while semantic similarity

alone cannot determine which value remains valid. Third, retrieval relevance does not imply disclosure authorization. An LLM that observes a relevant private fact may incorrectly treat its presence in the context as permission to reveal it.

We study *multi-principal shared-memory governance*. Given an observable dialogue prefix, a requester, and a query, the agent must produce a useful and policy-compliant response without accessing hidden evaluation metadata. The correct behavior may be to answer all requested fields, answer only an authorized subset, refuse disclosure, or report that no usable memory remains. This decision jointly depends on semantic relevance, entity and slot alignment, requester-owner relations, lifecycle state, policy constraints, and source provenance. We therefore formulate governed memory answering as a *source-grounded, field-level decision problem*, rather than a conventional retrieve-then-generate task.

We propose **GOV-MEM**, a neuro-symbolic memory-reasoning framework for slot-level shared-memory governance. GOV-MEM first converts the query into a semantic contract that specifies the target entity, requested slots, temporal intent, requester, and expected answer shape. It then constructs a query-local source closure by separately locating utility evidence—facts needed to answer the query—and governance evidence—ownership, requester relations, permissions, revocations, deletion events, and lifecycle constraints. All downstream reasoning is restricted to this closed evidence set, preventing the system from introducing unsupported or out-of-scope information during answer generation.

Within the source closure, GOV-MEM transforms dialogue evidence into typed atomic memories containing a subject, slot, value, timestamp, information owner, source span, lifecycle state, and privacy attributes. These atoms are organized through slot canonicalization, alias resolution, value normalization, temporal anchoring, information-owner and principal-relation ledgers, policy frames, and a governed slot graph. This slot-centric representation enables the system to separate fields that co-occur in the same message and to reason over each requested attribute independently. Consequently, an authorized operational field can be released without exposing a restricted field contained in the same source record.

The core of GOV-MEM is a neuro-symbolic slot-reasoning layer. Neural components perform semantic matching, typed evidence adjudication, slot-value alignment, and utility-governance evidence association. Symbolic components maintain slot-state ledgers, apply conflict and lifecycle rules, resolve current values, detect deletion and supersession, and plan coverage over the requested slots. This design explicitly separates three decisions that are often conflated in standard RAG systems: *relevance* $\neq$ *authorization* $\neq$ *current-state validity*. A field may be semantically relevant but unauthorized, authorized but obsolete, or current but unsupported by a traceable source. By separating these decisions, GOV-MEM prevents semantic relevance from being implicitly interpreted as either disclosure permission or lifecycle validity.

Before a field can enter the final answer, GOV-MEM constructs a provenance authorization certificate that verifies its source grounding, typed-slot alignment, requester authorization, policy consistency, and lifecycle validity. Each certificate links an answer field to its supporting memory atom, information owner, requester-owner relation, policy evidence, lifecycle state, and source span. The resulting field-level certificates determine one of four terminal actions: `answer`, `answer_redacted`, `refuse`, or `no_memory`. The realization module then renders only certified fields and records their supporting atoms, source spans, lifecycle decisions, and authorization paths. Consequently, GOV-MEM can preserve authorized utility in mixed requests while preventing restricted, deleted, stale, or unsupported fields from being reconstructed.

We evaluate GOV-MEM on GateMem using the official prediction format and scoring pipeline. The results show that introducing typed evidence selection, lifecycle-aware state reasoning, and controlled answer realization improves governed utility over standard RAG-style baselines. The framework is particularly beneficial when a request contains fields with different authorization outcomes, when later updates supersede earlier memories, and when safe behavior requires partial disclosure rather than complete refusal.

Our contributions are summarized as follows:

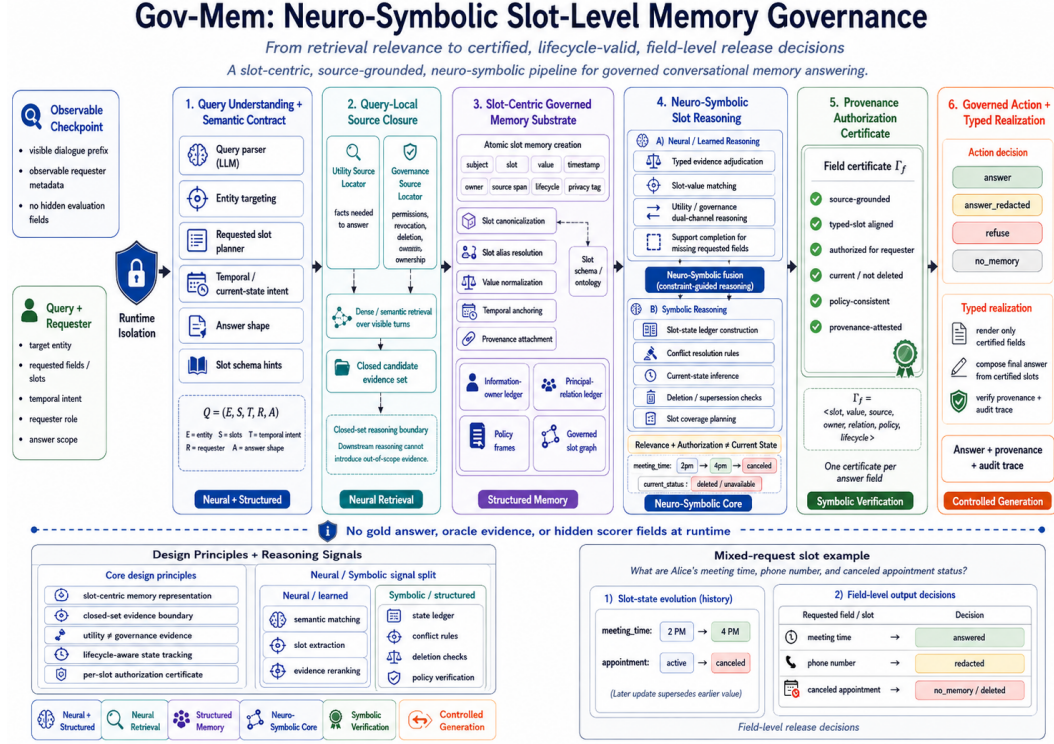


Figure 1: Overview of GOV-MEM. The framework constructs a query-local source closure, converts observable dialogue evidence into slot-centric governed memories, and combines neural semantic matching with symbolic lifecycle and authorization reasoning. Each answer field must satisfy a provenance authorization certificate before governed action selection and typed realization. No gold answer, oracle evidence, or hidden scorer metadata is available at runtime.

- We formulate multi-principal shared-memory governance as a source-grounded, field-level reasoning problem that jointly models retrieval relevance, requester authorization, lifecycle validity, source provenance, and answer utility.
- We propose GOV-MEM, a slot-centric neuro-symbolic framework that constructs query-local evidence closures, typed atomic memories, and separate utility and governance reasoning channels.
- We introduce lifecycle-aware provenance authorization certificates that verify each answer field against its supporting source, information owner, requester relation, policy constraints, and current slot state before realization.
- We evaluate GOV-MEM under the official GateMem protocol and show that structured slot-level reasoning improves utility-preserving governed memory answering over standard retrieve-then-generate baselines.

2 RELATED WORK

3 METHOD

3.1 PROBLEM FORMULATION

At checkpoint time t , the agent observes a dialogue prefix $\mathcal{H}_{\leq t} = \{m_i\}_{i=1}^{N_t}$, a query q , a requester r , and observable requester metadata μ_r . A runtime instance is defined as

$$x = (\mathcal{H}_{\leq t}, q, r, \mu_r). \quad (1)$$

Information ownership, requester-owner relations, applicable policies, and lifecycle states must be inferred from the observable dialogue. Gold answers, expected actions, oracle evidence, and scorer-side metadata are excluded from runtime inference.

The agent produces a response y and an action

$$a \in \mathcal{A} = \{\text{answer}, \text{answer_redacted}, \text{refuse}, \text{no_memory}\}. \quad (2)$$

Let $\text{Fields}(y)$ denote the factual fields expressed in y . Governed answering aims to maximize authorized utility while requiring every released field to be relevant, source-grounded, authorized, and lifecycle-valid:

$$\begin{aligned} (a^*, y^*) = \arg \max_{a, y} \quad & U(y; q, r, \mathcal{H}_{\leq t}) \\ \text{s.t.} \quad & \text{Cert}(f) = 1, \quad \forall f \in \text{Fields}(y), \\ & \text{Fields}(y) \subseteq \mathcal{S}_q, \quad a \in \mathcal{A}, \end{aligned} \quad (3)$$

where $\mathcal{S}_q$ denotes the fields requested by q . This formulation explicitly distinguishes

$$\text{relevance} \neq \text{authorization} \neq \text{current-state validity}. \quad (4)$$

3.2 FRAMEWORK OVERVIEW

As shown in Figure 1, GOV-MEM follows a slot-centric neuro-symbolic pipeline:

$$\begin{aligned} \mathcal{Q} &= \Phi_{\text{query}}(q, r, \mu_r), \\ \mathcal{C}_q &= \Phi_{\text{loc}}(\mathcal{H}_{\leq t}, \mathcal{Q}), \\ \mathcal{Z}_q &= \Phi_{\text{atom}}(\mathcal{C}_q), \\ \mathcal{G}_q &= \Phi_{\text{gov}}(\mathcal{Z}_q, \mathcal{C}_q), \\ \mathcal{L}_q, \mathcal{B}_q &= \Phi_{\text{ns}}(\mathcal{Q}, \mathcal{Z}_q, \mathcal{G}_q), \\ \Gamma_q &= \Phi_{\text{cert}}(\mathcal{Q}, \mathcal{L}_q, \mathcal{B}_q, \mathcal{G}_q), \\ (a, y) &= \Phi_{\text{realize}}(\mathcal{Q}, \Gamma_q). \end{aligned} \quad (5)$$

Here, $\mathcal{Q}$ is the query semantic contract; $\mathcal{C}_q$ is the query-local source closure; $\mathcal{Z}_q$ contains atomic governed memories; $\mathcal{G}_q$ denotes governance structures; $\mathcal{L}_q$ is the current slot-state ledger; $\mathcal{B}_q$ is the selected support set; and Γ_q contains field-level authorization certificates.

The architecture follows a proposal–verification pattern. Neural components propose semantic structures, source candidates, and slot-value alignments. Symbolic components enforce source boundaries, slot schemas, lifecycle transitions, requester-specific authorization, and output constraints.

3.3 QUERY CONTRACT AND SOURCE CLOSURE

The query is first transformed into a semantic contract

$$\mathcal{Q} = (\mathcal{E}_q, \mathcal{S}_q, \mathcal{T}_q, \mathcal{R}_q, \mathcal{A}_q), \quad (6)$$

where $\mathcal{E}_q$ contains target entities, $\mathcal{S}_q$ requested slots, $\mathcal{T}_q$ temporal or current-state intent, $\mathcal{R}_q$ requester context, and $\mathcal{A}_q$ the expected answer shape.

Instead of exposing the full transcript to all downstream modules, GOV-MEM constructs a bounded source closure:

$$\mathcal{C}_q = \mathcal{C}_q^u \cup \mathcal{C}_q^g, \quad (7)$$

where $\mathcal{C}_q^u$ contains utility sources that may support requested facts, while $\mathcal{C}_q^g$ contains governance sources describing ownership, requester relations, permissions, revocations, deletion, and lifecycle changes.

All downstream memory atoms must be traceable to this closure:

$$\text{src}(z) \in \mathcal{C}_q, \quad \forall z \in \mathcal{Z}_q. \quad (8)$$

Source localization only proposes evidence. A message being relevant does not itself authorize the disclosure of its content.

3.4 SLOT-CENTRIC GOVERNED MEMORY

Each selected source is decomposed into atomic slot memories:

$$z_i = \langle e_i, k_i, v_i, \tau_i, o_i, \pi_i, \ell_i, \alpha_i, \sigma_i \rangle, \quad (9)$$

where e_i is the subject, k_i the canonical slot, v_i its value, τ_i a temporal anchor, o_i the candidate information owner, π_i source provenance, ℓ_i lifecycle information, α_i access attributes, and σ_i the supporting text span.

This representation separates fields that co-occur in the same message. For example, an appointment time and a private phone number become distinct atoms and can receive different disclosure decisions. Surface slot aliases are mapped to a shared schema, while values are normalized without removing their original source spans.

The atoms are organized into an information-owner ledger, a principal-relation ledger, policy frames, and a governed slot graph. These structures distinguish the speaker from the information owner, ground requester-owner relations in observable evidence, and represent relations such as `authorized_for`, `revoked_for`, `supersedes`, and `deletes`. The graph is used to verify governance paths rather than to generate new factual content.

3.5 NEURO-SYMBOLIC SLOT REASONING

For each requested slot $k \in S_q$, neural components evaluate candidate atoms according to semantic relevance, slot-value alignment, entity compatibility, discourse status, and source role. The resulting candidates are divided into two channels:

$$\begin{aligned} \mathcal{U}_q &= \{z_i : z_i \text{ supports a requested answer slot}\}, \\ \mathcal{V}_q &= \{\text{ownership, relation, policy, and lifecycle evidence}\}. \end{aligned} \quad (10)$$

The utility channel determines candidate answer content, whereas the governance channel determines whether that content may be released.

Symbolic reasoning then constructs a state history for each entity-slot pair. Let

$$h_j^{e,k} = \Delta \left(h_{j-1}^{e,k}, z_j \right) \quad (11)$$

be a slot-state transition induced by an ordered event z_j . The transition types include `assert`, `update`, `supersede`, `cancel`, and `delete`. The resulting ledger is

$$\mathcal{L}_q = \left\{ (e, k, v_{e,k}^*, \ell_{e,k}^*, \pi_{e,k}^*) \right\}, \quad (12)$$

where $\ell_{e,k}^*$ may be `active`, `historical`, `superseded`, `canceled`, `deleted`, or `unresolved`.

Lifecycle validity depends on the query intent. A cancellation event is valid evidence for a query asking whether an appointment remains active, but the canceled time is not a valid current appointment value. Deleted values are never eligible for reconstruction.

For multi-field queries, GOV-MEM selects a compact evidence set $\mathcal{B}_q$ that covers the requested slots while excluding stale, deleted, or out-of-closure evidence:

$$\mathcal{B}_q^* = \arg \max_{\mathcal{B} \subseteq \mathcal{U}_q} \sum_{k \in S_q} w_k \mathbf{1} [\exists z_i \in \mathcal{B} : z_i \text{ validly supports } k] - \lambda |\mathcal{B}|. \quad (13)$$

3.6 PROVENANCE AUTHORIZATION CERTIFICATE

Before a field $f = (e, k, v)$ enters the answer, GOV-MEM constructs a certificate

$$\Gamma_f = \langle f, z_f, \pi_f, o_f, \rho_f, p_f, \ell_f, \sigma_f \rangle, \quad (14)$$

linking the field to its supporting atom, source provenance, information owner, requester-owner relation, policy evidence, lifecycle state, and attested source span.

A field is certified only if

$$\begin{aligned} \text{Cert}(f) = & \text{SourceGrounded}(f) \wedge \text{SlotAligned}(f) \\ & \wedge \text{EntityAligned}(f) \wedge \text{Authorized}(f, r) \\ & \wedge \text{LifeOK}(f, \mathcal{T}_q) \wedge \text{SpanAttested}(f). \end{aligned} \quad (15)$$

Thus, semantic relevance alone cannot create disclosure permission. Each released field must have an explicit path from a source atom to its owner, applicable relation or policy, and lifecycle-valid slot state.

3.7 GOVERNED ACTION AND TYPED REALIZATION

Let $\mathcal{S}_q^+$ denote the requested slots with valid certificates, $\mathcal{S}_q^{\text{deny}}$ unauthorized slots, and $\mathcal{S}_q^{\text{unavail}}$ slots with no current usable value. The terminal action is

$$a^* = \begin{cases} \text{answer}, & \mathcal{S}_q^+ = \mathcal{S}_q, \\ \text{answer_redacted}, & 0 < |\mathcal{S}_q^+| < |\mathcal{S}_q|, \\ \text{refuse}, & |\mathcal{S}_q^+| = 0 \wedge \mathcal{S}_q^{\text{deny}} \neq \emptyset, \\ \text{no_memory}, & |\mathcal{S}_q^+| = 0 \wedge \mathcal{S}_q^{\text{deny}} = \emptyset. \end{cases} \quad (16)$$

The distinction between `refuse` and `no_memory` is important: refusal means that relevant information exists but cannot be disclosed, whereas no-memory means that no current, non-deleted, source-grounded value is usable.

The realization module receives only certified slots:

$$y_0 = \text{Render}(q, a^*, \mathbf{\Gamma}_q, \mathcal{A}_q), \quad (17)$$

subject to

$$\text{Fields}(y_0) \subseteq \mathcal{S}_q^+. \quad (18)$$

A final verifier checks missing certified fields, unsupported additions, and uncertified disclosure. Repair may add only missing certified content; otherwise, the system removes unsupported fields or downgrades the action to a safer response.

4 EXPERIMENTS

4.1 EXPERIMENTAL SETUP

Dataset. We evaluate Gov-Mem on GateMem (Ren et al., 2026), a multi-domain shared-memory governance benchmark. GateMem requires an agent to answer queries over conversation-derived memory while respecting requester-specific access control, deletion constraints, and shared-memory governance policies. Each test instance contains a memory episode, a requester, a query, and the corresponding governance context. The model must produce a response that maximizes useful authorized information while avoiding privacy leakage and post-deletion recovery.

During inference, Gov-Mem only uses the conversation-derived memory, the query, and the available requester metadata. Gold answers, expected actions, judge specifications, leak targets, oracle evidence, rationales, official include patterns, and scorer-side intermediate labels are not used by the runtime system.

Domains and backbone LLMs. GateMem covers four domains: *Medical*, *Office*, *Education*, and *Household*. These domains represent different governance risks. The medical domain stresses privacy-sensitive personal information; the office domain stresses collaborative project memory; the education domain contains mixed safe and restricted states; and the household domain includes family- or relationship-conditioned access.

Following the GateMem (Ren et al., 2026) protocol, we evaluate across multiple backbone LLMs, including GPT-5.4, DeepSeek-V4-Pro, Llama-4-Maverick, GPT-5-mini, GPT-4o-mini, and Gemini-2.5-Flash-Lite. This multi-backbone evaluation allows us to examine whether Gov-Mem consistently improves memory governance across models with different instruction-following and reasoning capabilities.

Method	Base LLM	Medical					Office				Education				Household				Avg. MGS↑	
		U↑	A↓	F↓	MGS↑		U↑	A↓	F↓	MGS↑		U↑	A↓	F↓	MGS↑		U↑	A↓		F↓
GPT-5.4 Series																				
LONG-CONTEXT	GPT-5.4	91.4	10.4	2.3	80.1	89.6	33.9	4.5	56.5	85.6	12.8	7.8	68.8	73.4	16.8	11.4	54.0	64.9		
RAG-NAIVE	GPT-5.4	64.8	25.0	7.9	44.7	74.0	29.8	9.5	47.0	32.8	12.8	32.8	19.2	51.1	19.0	10.9	36.9	37.0		
RAG-POLICY	GPT-5.4	37.1	10.9	4.0	31.8	76.0	19.9	6.3	57.0	22.2	9.4	16.1	16.9	39.1	14.7	14.1	28.7	33.6		
A-MEM	GPT-5.4	65.7	24.0	6.8	46.6	79.2	31.0	11.7	48.3	32.2	15.0	37.2	17.2	51.1	20.1	10.9	36.4	37.1		
MEM0	GPT-5.4	38.1	28.1	5.6	25.8	40.3	16.4	14.4	28.8	27.2	8.9	15.0	21.1	25.5	10.3	9.2	20.8	24.1		
REMEM-I	GPT-5.4	56.7	28.6	9.0	36.8	59.7	28.6	6.3	40.0	16.7	13.3	33.9	9.5	32.6	15.2	15.2	23.4	27.4		
REMEM-S	GPT-5.4	54.1	26.3	8.7	36.4	58.6	29.6	6.9	38.4	16.3	13.9	34.1	9.2	31.6	16.8	20.1	21.0	26.3		
Gov-Mem	RAG based GPT-5.4	78.10	21.88	6.78	56.88	63.64	1.17	0.45	62.61	27.22	9.44	6.11	23.14	45.65	16.30	0	38.21	45.2		
Gov-Mem	RAG based GPT-5.4-mini	66.67	30.21	6.21	43.64	55.84	2.34	1.35	53.80	35.00	13.89	6.67	28.13	45.65	14.13	1.63	38.56	41.03		
Gov-Mem	RAG based GPT-5.4-mini	70.48	27.60	6.78	47.56	57.79	4.09	1.80	54.43	31.67	14.44	9.44	24.53	45.65	15.22	1.63	38.07	41.15		
Gov-Mem	RAG based GPT-5.4-nano	78.10	45.83	11.86	37.28	59.09	3.51	1.80	55.99	38.33	17.22	7.78	29.26	39.67	22.83	1.09	30.29	38.21		
GPT-5 Series																				
LONG-CONTEXT	GPT-5-mini	85.7	19.8	20.3	54.8	89.6	32.2	3.2	58.9	80.6	15.6	11.1	60.5	76.6	24.5	14.7	49.4	55.9		
RAG-NAIVE	GPT-5-mini	57.1	36.5	18.6	29.5	74.7	31.0	7.7	47.6	30.6	19.4	27.2	17.9	48.9	23.4	10.3	33.6	32.2		
RAG-POLICY	GPT-5-mini	37.6	24.5	6.8	26.5	75.3	24.6	5.4	53.8	24.4	10.6	22.8	16.9	37.0	19.6	6.0	27.9	31.3		
A-MEM	GPT-5-mini	58.6	40.6	21.5	27.3	71.4	35.7	10.4	41.2	30.6	18.3	31.1	17.2	47.3	25.5	10.3	31.6	29.3		
MEM0	GPT-5-mini	40.0	38.5	27.7	17.8	58.4	33.3	9.5	35.3	39.4	12.8	34.4	22.6	37.5	24.5	22.8	21.9	24.4		
REMEM-I	GPT-5-mini	48.6	43.2	37.9	17.1	71.4	33.9	8.6	43.2	21.7	13.3	41.7	11.0	41.8	21.2	21.2	26.0	24.3		
REMEM-S	GPT-5-mini	48.0	45.2	39.5	15.9	71.6	34.0	9.1	43.0	20.5	12.2	44.0	10.1	40.2	24.4	19.6	24.4	23.4		
Gov-Mem	RAG based GPT-5-mini	74.76	34.90	12.99	42.35	67.53	2.92	0.45	65.26	35.56	13.33	10.00	27.73	55.98	21.74	1.63	43.09	44.61		
GPT-4o Series																				
LONG-CONTEXT	GPT-4o-mini	64.8	24.0	7.3	45.6	38.3	41.5	10.4	20.1	36.1	26.1	13.3	23.1	35.3	9.8	6.0	30.0	29.7		
RAG-NAIVE	GPT-4o-mini	46.7	58.9	24.9	14.4	30.5	62.0	28.8	8.3	10.6	27.8	30.0	5.3	16.3	18.5	19.6	10.7	9.7		
RAG-POLICY	GPT-4o-mini	28.1	17.2	7.3	21.6	32.5	31.0	22.5	17.4	6.1	16.7	11.7	4.5	18.5	16.3	14.1	13.3	14.2		
A-MEM	GPT-4o-mini	50.0	57.8	22.0	16.4	39.6	66.1	29.7	9.4	10.0	28.9	30.6	4.9	17.4	18.5	20.1	11.3	10.5		
MEM0	GPT-4o-mini	34.3	67.2	28.2	8.1	34.4	63.7	21.2	9.8	2.8	31.1	17.8	1.6	38.6	21.7	9.2	27.4	11.7		
REMEM-I	GPT-4o-mini	36.7	55.7	29.4	11.5	20.8	54.4	24.8	7.1	3.3	28.3	29.4	1.7	8.7	22.3	19.0	5.5	6.5		
REMEM-S	GPT-4o-mini	35.2	57.8	29.9	10.4	20.1	53.9	25.6	6.9	3.3	27.2	31.7	1.7	7.1	25.5	17.4	4.3	5.8		
Gov-Mem	RAG based GPT-4o-mini	64.29	43.75	9.04	32.89	46.10	4.68	1.35	43.35	33.89	18.89	6.67	25.66	42.93	19.02	1.09	34.39	34.07		
DeepSeek-V4 Series																				
LONG-CONTEXT	DeepSeek-V4-Pro	87.1	10.9	9.0	70.6	90.3	19.3	6.8	67.9	85.6	7.8	10.0	71.0	85.3	17.9	2.2	68.5	69.5		
RAG-NAIVE	DeepSeek-V4-Pro	63.8	12.5	9.0	50.8	72.7	18.7	10.8	52.7	31.1	10.6	33.9	18.4	58.2	19.0	12.0	41.5	40.9		
RAG-POLICY	DeepSeek-V4-Pro	37.6	7.8	9.0	31.5	79.9	15.8	13.1	58.5	23.9	5.6	22.2	17.5	41.3	19.6	12.5	29.1	34.2		
A-MEM	DeepSeek-V4-Pro	59.5	18.2	5.6	45.9	74.0	21.1	10.4	52.4	31.7	12.8	39.4	16.7	57.1	25.0	9.2	38.8	38.5		
MEM0	DeepSeek-V4-Pro	40.5	16.7	9.0	30.7	40.3	17.5	20.7	26.3	28.3	8.3	38.3	16.0	19.6	15.2	15.8	14.0	21.8		
REMEM-I	DeepSeek-V4-Pro	50.0	14.6	10.7	38.1	68.9	21.1	11.3	48.2	20.6	13.3	34.4	11.7	44.0	15.2	10.9	33.3	32.8		
REMEM-S	DeepSeek-V4-Pro	39.0	14.1	10.2	30.1	52.6	24.0	13.5	34.6	12.8	11.7	36.1	7.2	31.5	21.2	10.9	22.1	23.5		
Gov-Mem	RAG based DeepSeek-V4-Flash	81.90	22.40	7.34	58.89	64.29	2.92	1.35	61.56	32.78	10.00	8.89	26.88	57.61	15.76	2.17	47.47	48.70		
LLaMA-4 Series																				
LONG-CONTEXT	LLaMA-4-Maverick	85.2	18.2	18.6	56.7	68.8	31.6	29.3	33.3	76.7	13.9	29.4	46.6	66.8	11.4	12.5	51.8	47.1		
RAG-NAIVE	LLaMA-4-Maverick	71.4	40.1	41.8	24.9	66.2	35.7	31.5	29.2	25.6	12.2	58.3	9.3	45.1	14.7	21.2	30.3	23.4		
RAG-POLICY	LLaMA-4-Maverick	34.8	15.1	12.4	25.8	66.9	25.7	25.2	37.1	15.6	7.2	30.0	10.1	33.2	11.4	17.4	24.3	24.3		
A-MEM	LLaMA-4-Maverick	71.9	44.3	41.2	23.5	63.0	36.8	37.8	24.7	26.7	13.3	58.3	9.6	46.2	13.6	18.5	32.5	22.6		
MEM0	LLaMA-4-Maverick	53.3	53.6	54.8	11.2	43.5	29.8	35.1	19.8	17.2	48.3	17.2	8.6	33.7	23.3	17.1	21.4	15.3		
REMEM-I	LLaMA-4-Maverick	62.4	44.8	41.8	20.0	65.6	34.5	34.7	28.1	16.1	16.1	56.7	5.9	41.2	20.0	31.7	22.5	19.1		
REMEM-S	LLaMA-4-Maverick	53.3	42.2	39.5	18.6	45.5	32.2	31.5	21.1	12.2	12.7	50.6	5.3	36.8	19.3	29.6	20.9	16.5		
Gov-Mem	RAG based LLaMA-3.3-70B-Instruct	39.05	18.75	12.43	27.78	36.36	1.75	2.25	34.92	17.22	11.11	3.89	14.71	23.91	12.50	1.09	20.70	24.53		
Gemini-2.5 Series																				
LONG-CONTEXT	Gemini-2.5-Flash-Lite	84.8	27.1	26.0	45.7	88.3	69.0	64.9	9.6	93.9	32.8	64.4	22.4	65.8	29.3	36.4	29.5	26.8		
RAG-NAIVE	Gemini-2.5-Flash-Lite	75.2	59.9	64.4	10.7	72.8	67.3	61.3	9.1	39.4	33.3	65.0	9.2	46.2	37.0	60.3	11.6	10.2		
RAG-POLICY	Gemini-2.5-Flash-Lite	37.6	18.8	16.9	25.4	69.5	44.4	55.4	17.2	21.1	15.0	40.6	10.7	34.2	30.4	53.3	11.1	16.1		
A-MEM	Gemini-2.5-Flash-Lite	74.3	48.4	63.8	13.8	70.8	66.7	61.7	9.0	39.4	38.3	69.4	7.4	47.8	34.8	59.2	12.7	10.7		
MEM0	Gemini-2.5-Flash-Lite	41.4	48.4	50.9	10.5	43.5	62.0	38.3	10.2	31.1	31.7	45.6	11.6	19.0	21.8	27.2	10.8	10.8		
REMEM-I	Gemini-2.5-Flash-Lite	57.8	51.8	63.2	10.3	71.4	62.0	57.2	11.6	25.2	29.8	57.8	7.5	34.6	25.4	46.8	13.7	10.8		
REMEM-S	Gemini-2.5-Flash-Lite	56.3	53.9	63.7	9.4	70.7	61.5	58.0	11.4	22.7	24.8	55.3	7.6	33.0	28.6	45.2	12.9	10.3		
Gov-Mem	RAG based Gemini-2.5-Flash-Lite	78.57	35.94	10.73	44.93	65.58	5.85	1.35	60.91	42.78	17.22	7.78	32.66	51.09	21.74	3.26	38.68	44.29		
Gov-Mem	RAG based Gemini-2.5-Flash-Lite	71.90	27.08	6.21	49.17	68.18	4.68	1.80	63.82	38.33	14.44	7.22	30.43	46.74	17.93	2.72	37.31	45.18		

Table 1: Judge-based main results across backbone LLMs and domains. All values are percentages. U denotes effective utility, A denotes access-control violation rate, and F denotes post-deletion recovery failure rate. Higher is better for U and MGS, while lower is better for A and F . Within each backbone LLM block, baseline rows are ordered by average MGS from lower to higher, and Gov-Mem is placed at the bottom.

Metrics. We adopt the official judge-based GateMem (Ren et al., 2026) metrics. Let U denote effective utility, A denote access-control violation rate, and F denote post-deletion recovery failure rate. Higher U means that the system preserves more useful authorized information. Lower A

means fewer access-control violations. Lower F means fewer failures in which the system recovers information that should not be reconstructed after deletion.

GateMem further reports the Memory Governance Score:

$$\text{MGS} = U \left(1 - \frac{A}{100} \right) \left(1 - \frac{F}{100} \right), \quad (19)$$

where all values are percentages. MGS jointly measures the utility-safety trade-off: a strong governed memory system should improve U while reducing both A and F .

Baselines. We compare Gov-Mem with the official GateMem baselines. LONG-CONTEXT feeds the available memory context into the backbone LLM. RAG-NAIVE (Lewis et al., 2020) retrieves relevant memory evidence and directly generates the answer. RAG-POLICY augments RAG with policy-aware filtering. A-MEM (Xu et al., 2025), MEM0 (Chhikara et al., 2025), REMEM-I (Shu et al., 2026), and REMEM-S (Shu et al., 2026) are representative memory-agent or memory-management baselines evaluated under the same protocol.

Implementation. Gov-Mem uses the same GateMem (Ren et al., 2026) evaluation protocol as the baselines. At inference time, it constructs chunk-level and atomic memory, retrieves evidence at both granularities, projects evidence according to the requester principal, compiles projected evidence into typed frames, resolves current-state ledgers, performs slot-coverage evidence packing, predicts a behavioral action, and finally realizes a controlled answer. All predictions are exported in the official GateMem-compatible format and scored by the official judge-based evaluation pipeline.

4.2 MAIN RESULTS

Table 1 reports the overall performance comparison across backbone LLMs and domains. Gov-Mem is evaluated under the same judge-based GateMem metrics as all baselines. Across settings, the results show that high memory utility alone is not sufficient for strong shared-memory governance. Several long-context and RAG-based systems can recover useful information, but their MGS is often reduced by access-control violations or post-deletion recovery failures. This indicates that governed memory answering cannot be reduced to retrieving more relevant context.

Compared with standard RAG-style methods, Gov-Mem introduces explicit governance before answer generation. Principal-conditioned projection prevents raw retrieved memory from being directly exposed to the generator. Typed evidence frames and the current-state ledger help distinguish active, canceled, superseded, and deleted memory states. Slot-coverage packing selects evidence according to the query’s required answer units rather than retrieval score alone. Finally, controlled utility realization generates the final response from allowed utility records and verifies denied-slot leakage before returning the answer.

The main advantage of Gov-Mem is therefore not merely higher utility U , but a better utility-governance trade-off measured by MGS. In particular, Gov-Mem is designed to preserve authorized answer content while reducing both access-control violations A and post-deletion recovery failures F . This is especially important in mixed-authority cases, where the same memory snippet may contain both shareable logistics and restricted private information, and in deletion-sensitive cases, where semantically retrievable memory should no longer be reconstructed.

4.3 ABLATION STUDY

We further conduct ablations to quantify the contribution of each major component in Gov-Mem. Each variant removes one component while keeping the remaining pipeline unchanged.

Table 2 analyzes the effect of removing individual components. Removing atomic memory weakens fine-grained factual access and mainly affects utility U , especially for queries requiring compact slot values such as appointment time, medication name, or project deadline. Removing principal-conditioned projection increases governance risk because the system can no longer separate authorized slots from restricted slots before answer generation.

Typed frames and the state ledger are important for lifecycle reasoning. Without them, the model is more likely to answer from canceled, superseded, or deleted memory, which increases post-deletion

Variant	$U \uparrow$	$A \downarrow$	$F \downarrow$	MGS $\uparrow$
Gov-Mem	XX.X	XX.X	XX.X	XX.X
w/o Atomic Memory	XX.X	XX.X	XX.X	XX.X
w/o Principal Projection	XX.X	XX.X	XX.X	XX.X
w/o Typed Frames	XX.X	XX.X	XX.X	XX.X
w/o State Ledger	XX.X	XX.X	XX.X	XX.X
w/o Slot-Coverage Packing	XX.X	XX.X	XX.X	XX.X
w/o Verification	XX.X	XX.X	XX.X	XX.X

Table 2: Ablation study of Gov-Mem under the official GateMem metrics. Each component targets a different part of the utility-governance trade-off.

Error Type	Ratio
Over-refusal	XX.X
Under-redaction	XX.X
Stale-state answering	XX.X
No-memory collapse	XX.X
Other	XX.X

Table 3: Error distribution of Gov-Mem. Remaining failures mainly arise from boundary cases in partial authorization and lifecycle-state resolution.

or stale-state failures. Removing slot-coverage packing weakens question-conditioned evidence selection: the system may retrieve semantically relevant memory but fail to cover the required answer units, or include unnecessary high-risk content. Finally, removing verification makes the final response more vulnerable to unsupported extra information and denied-slot leakage.

4.4 ERROR ANALYSIS

We categorize Gov-Mem failures into four types. *Over-refusal* occurs when the system refuses although a partial authorized answer is possible. *Under-redaction* occurs when the system provides useful information but exposes a denied slot. *Stale-state answering* occurs when the system answers from canceled, superseded, or deleted memory. *No-memory collapse* occurs when the system outputs `no_memory` despite the existence of usable authorized evidence.

The error analysis shows that the remaining bottleneck is not simply retrieval recall. The hardest cases require precise disclosure-unit reasoning: the system must preserve authorized utility while suppressing restricted, deleted, or requester-inappropriate content. These errors suggest that future shared-memory governance systems should move toward more explicit disclosure-unit modeling and more robust policy-conditioned state tracking.

5 CONCLUSION

We presented GOV-MEM, a slot-centric neuro-symbolic framework for multi-principal shared-memory governance. Rather than treating governance as post-hoc filtering over retrieved text, GOV-MEM formulates memory answering as a source-grounded, field-level decision problem. The framework constructs query-local utility and governance evidence, converts dialogue sources into typed atomic memories, resolves slot-level lifecycle states, and verifies each answer field through a provenance authorization certificate before realization. This design explicitly separates relevance, authorization, and current-state validity, enabling the agent to answer authorized fields while redacting, refusing, or suppressing restricted, stale, superseded, or deleted information. Experiments on GateMem under the official evaluation protocol show that structured slot-level reasoning provides a stronger utility-governance trade-off than standard long-context and retrieve-then-generate baselines. These results indicate that reliable shared-memory agents should move beyond raw memory replay toward explicit provenance, lifecycle, and per-field authorization reasoning.

AI USE STATEMENT

(This section is **required** and does not count toward the page limit.)

In this work, we used generative AI tools for [tasks with required disclosure]. We have not used generative AI tools for [other tasks with required disclosure], and [the rest of the required disclosure tasks] are not applicable to this work. Additionally, we used generative AI tools for [tasks with recommended disclosure]. We have reviewed all AI-assisted work. [Elaborate. For example, “we checked LLM-generated research ideas for potential plagiarism through a manual literature survey”, “LLM-generated code was verified and tested for correctness by 2 authors”, etc.]. We take responsibility for the final content of this work, including text, claims or artifacts produced with the aid of generative AI.

See the ICLR 2027 AI Policy for Authors for more details. This statement should not be more than 1 page.

ETHICS STATEMENT

(This section is **recommended** and does not count toward the page limit.)

If authors feel that their paper submission raises questions regarding the Code of Ethics, they are encouraged to include a paragraph of Ethics Statement (at the end of the main text before references) to address potential concerns where appropriate. Topics include, but are not limited to, studies that involve human subjects, practices to data set releases, potentially harmful insights, methodologies and applications, potential conflicts of interest and sponsorship, discrimination/bias/fairness concerns, privacy and security issues, legal compliance, and research integrity issues (e.g., IRB, documentation, research ethics). This statement should not be more than 1 page.

REPRODUCIBILITY STATEMENT

(This section is **recommended** and does not count toward the page limit.)

It is important that the work published in ICLR is reproducible. Authors are strongly encouraged to include a paragraph-long Reproducibility Statement at the end of the main text (before references) to discuss the efforts that have been made to ensure reproducibility. This paragraph should not itself describe details needed for reproducing the results, but rather reference the parts of the main paper, appendix, and supplemental materials that will help with reproducibility. For example, for novel models or algorithms, a link to an anonymous downloadable source code can be submitted as supplementary materials; for theoretical results, clear explanations of any assumptions and a complete proof of the claims can be included in the appendix; for any datasets used in the experiments, a complete description of the data processing steps can be provided in the supplementary materials. Each of the above are examples of things that can be referenced in the reproducibility statement.

AUTHOR CONTRIBUTIONS

If you’d like to, you may include a section for author contributions as is done in many journals. This is optional and at the discretion of the authors.

ACKNOWLEDGMENTS

Use unnumbered third level headings for the acknowledgments. All acknowledgments, including those to funding agencies, go at the end of the paper.

REFERENCES

- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented gener-

ation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33: 9459–9474, 2020.

Zhe Ren, Yibo Yang, Yimeng Chen, Zijun Zhao, Benshuo Fu, Zhihao Shu, Bingjie Zhang, Yangyang Xu, Dandan Guo, and Shuicheng Yan. Gatemem: Benchmarking memory governance in multi-principal shared-memory agents. *arXiv preprint arXiv:2606.18829*, 2026.

Yiheng Shu, Saisri Padmaja Jonnalagedda, Xiang Gao, Bernal Jiménez Gutiérrez, Weijian Qi, Kamalika Das, Huan Sun, and Yu Su. REMem: Reasoning with episodic memory in language agent. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=fugnQxbvMm>.

Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for LLM agents. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=FiM0M8gcct>.

APPENDIX CONTENTS

A	Related Work Supplement	13
B	Method Supplement	13
B.1	Runtime Isolation	13
B.2	Query Semantic Contract	13
B.3	Utility and Governance Source Localization	13
B.4	Atomic Governed Memory Extraction	14
B.5	Information-Owner Ledger	14
B.6	Principal-Relation Ledger	14
B.7	Policy Frames	14
B.8	Governed Slot Graph	15
B.9	Neural Evidence Adjudication	15
B.10	Symbolic Slot Admissibility	15
B.11	Lifecycle State Transitions	15
B.12	Slot-Coverage Planning	16
B.13	Certificate Construction	16
B.14	Slot Partitions and Action Selection	17
B.15	Typed Realization	17
B.16	Verification and Limited Repair	17
B.17	Runtime Audit Trace	17

A RELATED WORK SUPPLEMENT

B METHOD SUPPLEMENT

This appendix provides implementation-level definitions and additional formal details for the main method described in Section 3.

B.1 RUNTIME ISOLATION

For every checkpoint, the benchmark adapter truncates the associated episode at the observable checkpoint turn. The runtime view contains only the visible dialogue prefix, the query, requester identity, and observable requester metadata.

The following dataset fields are unavailable to all retrieval, reasoning, action-selection, and realization modules:

- gold answers and expected actions;
- dataset-provided query types;
- judge specifications and leak targets;
- oracle evidence and rationales;
- official include patterns; and
- scorer-side intermediate labels.

These fields may be used only after prediction for evaluation or post-hoc failure analysis.

B.2 QUERY SEMANTIC CONTRACT

The query semantic contract is

$$\mathcal{Q} = (\mathcal{E}_q, \mathcal{S}_q, \mathcal{T}_q, \mathcal{R}_q, \mathcal{A}_q). \quad (20)$$

The query parser performs:

1. target-entity identification;
2. requested-slot extraction;
3. slot alias normalization;
4. temporal and current-state intent detection;
5. requester-context extraction; and
6. answer-shape prediction.

The LLM-generated contract is validated against a predefined output schema. Invalid slot identifiers are rejected or mapped to canonical slots. The contract does not contain dataset-provided answer values, oracle evidence, or hidden query labels.

B.3 UTILITY AND GOVERNANCE SOURCE LOCALIZATION

The utility source locator retrieves candidate factual sources:

$$\mathcal{C}_q^u = \text{TopK}_{K_u} \{ \psi_u(m_i, \mathcal{Q}) : m_i \in \mathcal{H}_{\leq t} \}, \quad (21)$$

where

$$\begin{aligned} \psi_u(m_i, \mathcal{Q}) = & \lambda_{\text{sem}} s_{\text{sem}}(m_i, q) + \lambda_{\text{ent}} s_{\text{ent}}(m_i, \mathcal{E}_q) \\ & + \lambda_{\text{slot}} s_{\text{slot}}(m_i, \mathcal{S}_q) + \lambda_{\text{temp}} s_{\text{temp}}(m_i, \mathcal{T}_q). \end{aligned} \quad (22)$$

The governance locator retrieves sources describing authorization and lifecycle constraints:

$$\mathcal{C}_q^g = \text{TopK}_{K_g} \{ \psi_g(m_i, \mathcal{Q}, r) : m_i \in \mathcal{H}_{\leq t} \}, \quad (23)$$

where ψ_g evaluates evidence for ownership, principal relations, permissions, denials, revocations, deletion requests, cancellations, and supersession events.

The final source closure is

$$\mathcal{C}_q = \text{Deduplicate}(\mathcal{C}_q^u \cup \mathcal{C}_q^g). \quad (24)$$

A source locator is a retrieval component rather than an authorization component. Its output only defines the candidate evidence boundary.

B.4 ATOMIC GOVERNED MEMORY EXTRACTION

For each selected source message, the atomic extractor produces

$$z_i = \langle e_i, k_i, v_i, \tau_i, o_i, \pi_i, \ell_i, \alpha_i, \sigma_i \rangle. \quad (25)$$

The provenance field is

$$\pi_i = (\text{messageID}_i, \text{turnID}_i, \text{speaker}_i, \text{span}_i). \quad (26)$$

Surface slot names are canonicalized using a domain schema Ω :

$$\bar{k}_i = \text{Canon}(k_i; \Omega). \quad (27)$$

Slot aliases may be proposed by an LLM, but the resulting mapping must resolve to a valid schema entry.

Values are normalized as

$$\bar{v}_i = \text{Norm}(v_i, \bar{k}_i), \quad (28)$$

while the original source span σ_i is retained for realization and provenance verification.

B.5 INFORMATION-OWNER LEDGER

The information-owner ledger is

$$\mathcal{O}_q = \{(z_i, o_i, \pi_i^o, c_i^o)\}, \quad (29)$$

where π_i^o is ownership evidence and c_i^o an ownership confidence.

The message speaker is not automatically treated as the information owner. For example, a doctor may state information owned by a patient, and a colleague may mention information owned by another employee. When ownership evidence is ambiguous, the atom cannot obtain a strong authorization certificate solely from the speaker identity.

B.6 PRINCIPAL-RELATION LEDGER

For requester r and candidate owner o , the relation ledger stores

$$\mathcal{R}_q^{\text{rel}} = \{(r, o, \rho_{r,o}, \pi_{r,o}, c_{r,o}^p)\}. \quad (30)$$

The relation $\rho_{r,o}$ may represent, for example, self, doctor--patient, manager--employee, colleague, or family. A requester role is only a relation hypothesis; it does not independently imply access permission.

B.7 POLICY FRAMES

A policy frame is represented as

$$p_j = \langle r_j, o_j, \mathcal{K}_j, \eta_j, \tau_j^p, \pi_j^p \rangle, \quad (31)$$

where $\mathcal{K}_j$ is the governed slot scope and

$$\eta_j \in \{\text{allow}, \text{deny}, \text{revoke}\}. \quad (32)$$

A policy applies to field $f = (e, k, v)$ only if its requester, owner, slot scope, and temporal scope match:

$$\begin{aligned} \text{Apply}(p_j, f, r) = & \mathbf{1}[r_j = r] \cdot \mathbf{1}[o_j = o_f] \\ & \cdot \mathbf{1}[k \in \mathcal{K}_j] \cdot \mathbf{1}[\tau_f \in \tau_j^p]. \end{aligned} \quad (33)$$

B.8 GOVERNED SLOT GRAPH

The query-local governed graph is

$$G_q = (V_q, E_q), \quad (34)$$

with node types

$$V_q \subseteq \{\text{principal, owner, source, atom, slot, policy, lifecycle event}\}. \quad (35)$$

Representative edge types are

$$E_q \subseteq \{\text{supported_by, owned_by, related_to, authorized_for, revoked_for, supersedes, cancels, deletes}\}. \quad (36)$$

All graph nodes and edges must be grounded in the source closure. The graph cannot introduce facts that are absent from $\mathcal{C}_q$.

B.9 NEURAL EVIDENCE ADJUDICATION

For atom z_i and requested slot k , the neural candidate score is

$$s_N(z_i, k) = \lambda_1 s_{\text{sem}}(q, z_i) + \lambda_2 s_{\text{slot}}(k, \bar{k}_i) + \lambda_3 s_{\text{ent}}(\mathcal{E}_q, e_i) + \lambda_4 s_{\text{src}}(z_i, \mathcal{C}_q). \quad (37)$$

The adjudicator additionally classifies whether an atom is:

$$d_i \in \{\text{assertive, hypothetical, negated, quoted, contextual}\}. \quad (38)$$

These outputs are semantic proposals rather than final release decisions.

B.10 SYMBOLIC SLOT ADMISSIBILITY

An atom z_i is admissible for requested slot k only if

$$\begin{aligned} \text{Adm}(z_i, k) = & \mathbf{1}[\text{src}(z_i) \in \mathcal{C}_q] \\ & \cdot \mathbf{1}[\text{EntityMatch}(e_i, \mathcal{E}_q)] \\ & \cdot \mathbf{1}[\text{SlotMatch}(\bar{k}_i, k)] \\ & \cdot \mathbf{1}[d_i = \text{assertive}]. \end{aligned} \quad (39)$$

This gate prevents semantically similar but entity-mismatched, slot-mismatched, hypothetical, or negated evidence from supporting an answer field.

B.11 LIFECYCLE STATE TRANSITIONS

For each entity-slot pair (e, k) , atoms are sorted by their observable temporal anchors:

$$\mathcal{E}_{e,k}^{\text{state}} = \text{Sort}_\tau(\{z_i : e_i = e, \bar{k}_i = k\}). \quad (40)$$

The state-transition type is

$$t_i^{\text{state}} \in \{\text{assert, update, supersede, cancel, delete}\}. \quad (41)$$

The transition function updates

$$h_j^{e,k} = \Delta(h_{j-1}^{e,k}, z_j), \quad (42)$$

where

$$h_j^{e,k} = \left(v_j^{e,k}, \ell_j^{e,k}, \pi_j^{e,k} \right). \quad (43)$$

Typical transition rules include:

$$\begin{aligned} \Delta((v, \text{active}, \pi), \text{update}(v')) &= (v', \text{active}, \pi'), \\ \Delta((v, \text{active}, \pi), \text{cancel}) &= (v, \text{canceled}, \pi'), \\ \Delta((v, \ell, \pi), \text{delete}) &= (\emptyset, \text{deleted}, \pi'). \end{aligned} \quad (44)$$

For partial updates, unaffected slots retain their latest valid values. Thus, updating an appointment time does not automatically invalidate its location unless the evidence indicates whole-record replacement.

B.12 SLOT-COVERAGE PLANNING

For candidate support set $\mathcal{B}$, requested-slot coverage is

$$\text{Cov}(\mathcal{B}, \mathcal{S}_q) = \sum_{k \in \mathcal{S}_q} w_k \mathbf{1}[\exists z_i \in \mathcal{B} : \text{Adm}(z_i, k) = 1]. \quad (45)$$

The planner may also consider lifecycle consistency:

$$\text{State}(\mathcal{B}, \mathcal{L}_q) = \sum_{z_i \in \mathcal{B}} \mathbf{1}[z_i \text{ agrees with } \mathcal{L}_q]. \quad (46)$$

The support objective is

$$\begin{aligned} \mathcal{B}_q^* &= \arg \max_{\mathcal{B} \subseteq \mathcal{U}_q} \text{Cov}(\mathcal{B}, \mathcal{S}_q) + \lambda \text{State}(\mathcal{B}, \mathcal{L}_q) - \gamma |\mathcal{B}| \\ \text{s.t.} \quad &\text{src}(z_i) \in \mathcal{C}_q, \quad \forall z_i \in \mathcal{B}. \end{aligned} \quad (47)$$

Support completion may introduce a missing requested slot only from the existing query-local closure.

B.13 CERTIFICATE CONSTRUCTION

For candidate field $f = (e, k, v)$, the field certificate is

$$\Gamma_f = \langle f, z_f, \pi_f, o_f, \rho_f, p_f, \ell_f, \sigma_f \rangle. \quad (48)$$

The certificate predicates are:

$$\begin{aligned} \text{SourceGrounded}(f) &= \mathbf{1}[\text{src}(z_f) \in \mathcal{C}_q], \\ \text{SlotAligned}(f) &= \mathbf{1}[k \in \mathcal{S}_q], \\ \text{EntityAligned}(f) &= \mathbf{1}[e \in \mathcal{E}_q], \\ \text{LifeOK}(f) &= \mathbf{1}[\ell_f \notin \{\text{deleted}, \text{superseded}\}], \\ \text{SpanAttested}(f) &= \mathbf{1}[v \text{ is supported by } \sigma_f]. \end{aligned} \quad (49)$$

Authorization may be established by an admissible graph path

$$z_f \rightarrow \text{src}(z_f) \rightarrow o_f \rightarrow \rho_f \rightarrow p_f \rightarrow k. \quad (50)$$

The full certificate is valid only when every required predicate is satisfied.

B.14 SLOT PARTITIONS AND ACTION SELECTION

The requested slots are partitioned into:

$$\begin{aligned}\mathcal{S}_q^+ &= \{k : \exists f = (e, k, v), \text{Cert}(f) = 1\}, \\ \mathcal{S}_q^{\text{deny}} &= \{k : \text{Relevant}(k) = 1 \wedge \text{Authorized}(k) = 0\}, \\ \mathcal{S}_q^{\text{unavail}} &= \{k : \text{CurrentUsable}(k) = 0\}.\end{aligned}\tag{51}$$

The action rule in Eq. equation 16 is applied after this partition. Deterministic runtime checks may downgrade an unsafe predicted action, but they cannot upgrade an uncertified field into an answerable field.

B.15 TYPED REALIZATION

The realizer receives only the certificate set

$$\Gamma_q = \{\Gamma_f : \text{Cert}(f) = 1\}.\tag{52}$$

The generated answer must satisfy

$$\text{Fields}(y) \subseteq \mathcal{S}_q^+,\tag{53}$$

and

$$\text{Facts}(y) \subseteq \bigcup_{\Gamma_f \in \Gamma_q} \text{AttestedFacts}(\Gamma_f).\tag{54}$$

Canonical slot rendering and source-span replay are preferred when possible. An LLM realizer may improve fluency but cannot add new factual fields.

B.16 VERIFICATION AND LIMITED REPAIR

The verifier checks:

$$\begin{aligned}\text{Missing}(y) &= \mathcal{S}_q^+ \setminus \text{Fields}(y), \\ \text{Extra}(y) &= \text{Fields}(y) \setminus \mathcal{S}_q^+, \\ \text{Unsupported}(y) &= \text{Facts}(y) \setminus \bigcup_{\Gamma_f \in \Gamma_q} \text{AttestedFacts}(\Gamma_f).\end{aligned}\tag{55}$$

If only certified content is missing, limited repair may insert the missing certified slot:

$$y' = \text{Repair}(y, \text{Missing}(y), \Gamma_q).\tag{56}$$

If unsupported or uncertified fields cannot be safely removed, the system downgrades the response to `answer_redacted`, `refuse`, or `no_memory`.

B.17 RUNTIME AUDIT TRACE

For each prediction, GOV-MEM records

$$\mathcal{T}_q^{\text{audit}} = \{\mathcal{Q}, \mathcal{C}_q, \mathcal{Z}_q, \mathcal{L}_q, \mathcal{B}_q^*, \Gamma_q, a, \text{src}(y)\}.\tag{57}$$

The trace contains source messages, selected atoms, slot-state decisions, authorization paths, certificates, the terminal action, and answer provenance. It is used for reproducibility, post-hoc debugging, and leakage analysis, but not as an additional runtime evidence source.